

Letter Recognition Using ANNs

Kristina Bhandari¹, Namita Bhatta²

¹Institute Of Engineering, Thapathali Campus, Bagmati 446000 Nepal (e-mail: christinabhandari60@gmail.com)

Corresponding author: Kristina Bhandari(e-mail:christinabhandari60@gmail.com).

ABSTRACT Artificial Neural Networks (ANNs) are powerful computational models that mimic the workings of the human brain to recognize patterns and make predictions. In this study, we use ANNs with the MNIST and Letter Recognition datasets, containing 60,000 and 20,000 samples, respectively. We examine how different initialization methods, such as random and Kaiming, and regularization techniques affect the network's accuracy. By testing activation functions like Leaky ReLU, tanh, and sigmoid, we find that while activation functions and weight initialization improved accuracy, dropout regularization caused a significant drop which highlights the importance of careful technique selection and tuning.

INDEX TERMS Artificial Neural Networks (ANNs), Letter Recognition, MNIST, Activation functions, Regularization

I. INTRODUCTION

Artificial Neural Networks (ANNs) are a sophisticated category of machine learning models designed to mimic the neural structure of the human brain. Composed of interconnected nodes, or "neurons," arranged in multiple layers, ANNs have the ability to learn intricate patterns and relationships from data. These networks excel in various tasks, including image and speech recognition, natural language processing, and more. The learning process in ANNs involves training, during which the network adjusts its internal parameters, or weights, to minimize prediction errors. This adaptability allows ANNs to generalize well to new data, making them effective for tasks such as classification, regression, and generative modeling. The layered architecture of ANNs, consisting of input, hidden, and output layers, is crucial for their ability to transform raw data into meaningful predictions through progressively complex representations.

ANNs have a layered architecture: the input layer receives raw data, hidden layers extract and learn features, and the output layer generates predictions. Neurons in hidden layers process

information through weighted connections and activation functions, which introduce non-linearity to capture intricate patterns. Training an Artificial Neural Network (ANN) involves forward propagation and backward propagation processes. During forward propagation, input data passes through the network's layers, being transformed by weighted connections and activation functions to produce predictions. Backward propagation, or backpropagation, follows by calculating gradients of the loss function with respect to network weights. This step is essential for understanding how each weight affects the overall error, facilitating weight adjustments to minimize the loss. Gradient descent, an optimization technique, uses these gradients to iteratively update the weights in the direction that reduces the error. These interconnected processes are central to training neural networks, allowing them to learn and improve their performance over successive iterations.

Activation functions in ANNs introduce non-linearity, enabling the network to capture complex patterns. They determine neuron output based on the weighted sum of inputs. Common functions

include sigmoid, tanh, and ReLU, each with unique properties. Sigmoid and tanh compress values into ranges, while ReLU accelerates training by addressing the vanishing gradient problem. The choice of activation function impacts training speed, convergence, and performance. Weight initialization methods, like Kaiming and Xavier, are crucial for effective ANN training. Xavier maintains consistent variance across layers, while Kaiming prevents the "dying ReLU" problem. These methods enhance ANN convergence and performance by mitigating gradient issues. Thus, by adjusting connection strengths during training, ANNs become adaptable, enabling accurate predictions on new data by generalizing from previous examples. This capability, combined with their ability to identify complex non-linear correlations, has driven advancements in various fields and established ANNs as a key component of contemporary machine learning.

II. Literature review

The paper [1] explores the advantages of ANNs in medical science, particularly gastroenterology. The authors highlight ANNs' ability to handle complex, nonlinear data through adaptive learning and dynamic weight adjustments. ANNs are praised for their pattern recognition, problem-solving in nonlinear scenarios, and robust generalization capabilities. The article emphasizes ANNs' potential to integrate diverse clinical data types, improving diagnostic accuracy and patient outcomes. Grossi and Buscema conclude that ANNs offer significant advancements in medical data analysis and patient care.

In [2] the perceptron was introduced, an early artificial neural network model designed to simulate the brain's mechanisms for information processing and recognition. The paper addresses key questions about sensory information detection, storage, and influence on behavior, favoring a connectionist approach over representational memory theories. Rosenblatt's model emphasizes probabilistic neural connections and their adaptation through learning, drawing analogies

between biological neural systems and machine learning. The perceptron, characterized by its layers of sensory and association units, showcases the potential for neural networks to perform tasks such as pattern recognition and decision-making through adaptive, probabilistic connections, laying foundational concepts for modern artificial intelligence and machine learning.

The study reported in the paper [2] presents the foundational concept of backpropagation in neural networks. The paper highlighted the importance of non-linear activation functions and the role of hidden layers in capturing intricate data representations. This method is pivotal for training multilayer neural networks by minimizing the error rate. It works by propagating the error from the output layer back through the network, and adjusting the weights to reduce discrepancies between the predicted and actual outputs. It also addressed challenges such as the vanishing gradient problem, which can impede the training of deep networks, and proposed solutions to mitigate this issue. This approach enabled the development of more complex and accurate models in machine learning, significantly influencing the field of artificial neural networks.

According to [4], the choice of activation function significantly impacts the performance of neural networks, and there is no one-size-fits-all solution. This survey paper provides a comprehensive review of various activation functions used in neural networks, including ReLU, Sigmoid, Tanh, and more recent functions like Swish and Mish. The paper emphasizes the importance of choosing appropriate activation functions to improve the training dynamics and performance of neural networks. It explores the relationship between activation functions and weight initialization methods, particularly Xavier and He Normal initialization, highlighting how these methods complement specific activation functions to enhance convergence and stability during training. It concludes that newer activation functions like Swish and Mish can offer superior performance over traditional functions in certain scenarios. It

concludes that the choice of activation function significantly impacts the performance of neural networks, and there is no one-size-fits-all solution.

III. METHODOLOGY

A. Theory

The methodology of an Artificial Neural Network (ANN) begins with crafting a well-defined network architecture, encompassing the arrangement and number of layers and neurons. During forward propagation, input data is passed through the network, where each neuron computes a weighted sum of its inputs, adds a bias, and applies an activation function to produce an output. The loss function measures the difference between the predicted output and the actual target. Backpropagation then calculates the gradient of the loss with respect to each weight, propagating the error backward through the network. These gradients are used to update the weights and biases using optimization algorithms like Gradient Descent. This process is repeated over multiple epochs, refining the model's parameters to improve its performance and accuracy.

B. Working Principle

Neural Network

A neural network is a computational model inspired by the way biological neural networks in the human brain process information. It consists of layers of interconnected nodes (or neurons), where each connection has an associated weight.

For a single neuron in a neural network, the output y is computed using the formula:

$$y = \sigma \left(\sum_{i=1}^n w_i x_i + b \right)$$

where σ is an activation function,

w_i are weights,

x_i are inputs,

b is a bias.

For multiple neurons and layers, the formula is:

$$y = \sigma(WX + b)$$

Forward Propagation

Forward propagation is the process of passing input data through a neural network to obtain the output. During forward propagation, each neuron in the network computes a weighted sum of its inputs, adds a bias, and then applies an activation function to produce its output. This output serves as input for the next layer, continuing until the final output layer is reached.

$$Z^{[1]} = W^{[1]}x + b^{[1]}$$

$$A^{[1]} = \text{Relu}(Z^{[1]})$$

$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$$

$$A^{[2]} = \text{Softmax}(Z^{[2]})$$

Backpropagation

Backpropagation is a training algorithm for neural networks that minimizes error by adjusting weights. It involves computing the error between predicted and actual outputs, then propagating this error backward through the network to update weights using the gradient of the loss function. This process helps the network learn by refining its predictions.

$$\frac{\partial \text{cost}}{\partial Z^{[2]}} = A^{[2]} - Y$$

$$\Delta Z^{[2]} = A^{[2]} - Y$$

$$\Delta W^{[2]} = \frac{\partial \text{cost}}{\partial W^{[2]}} = \frac{1}{m} \Delta Z^{[2]} A^{[1]T}$$

$$\Delta b^{[2]} = \frac{\partial \text{cost}}{\partial b^{[2]}} = \frac{1}{m} \sum \Delta Z^{[2]}$$

$$\frac{\partial \text{cost}}{\partial Z^{[1]}} = \Delta Z^{[1]} = W^{[2]T} \Delta Z^{[2]} \cdot g'_{\text{ReLU}}(Z^{[1]})$$

$$\Delta W^{[1]} = \frac{\partial \text{cost}}{\partial W^{[1]}} = \frac{1}{m} \Delta Z^{[1]} [A^{[0]}]^T$$

Parameter Update

The weights and biases of a neural network are updated to minimize the error between the

predicted output and the actual target. This is done during training using optimization algorithms like Stochastic Gradient Descent (SGD).

$$W^{[2]} = W^{[2]} - \alpha dW^{[2]}$$

$$b^{[2]} = b^{[2]} - \alpha db^{[2]}$$

$$W^{[1]} = W^{[1]} + \alpha dW^{[1]}$$

$$b^{[1]} = b^{[1]} - \alpha db^{[1]}$$

Where α is the learning rate, $W^{[i]}$ are the weight matrices, and $b^{[i]}$ are the bias vectors for each layer

Activation Functions

Activation functions in artificial neural networks introduce non-linearity into the model, allowing it to learn complex patterns. Common activation functions include:

Sigmoid Activation: It maps input values to a range between 0 and 1, making it useful for binary classification problems. However, it can suffer from vanishing gradient issues where gradients become very small during backpropagation, slowing down learning.

$$g_{\text{sigmoid}}(z) = \frac{1}{1 + e^{-z}}$$

1) Tanh Activation

It maps input values to a range between -1 and 1. It is zero-centered, which can help with convergence during training. Like the sigmoid function, it can also suffer from vanishing gradients.

$$g_{\text{tanh}}(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

2) Softmax Function

It Converts logits into probabilities, making it useful for multi-class classification problems. It ensures that the output values are in a probability distribution.

$$\text{Softmax}(z_i) = \frac{e^{z_j}}{\sum_j e^{z_j}}$$

3) ReLU Activation

ReLU Outputs the input value directly if it's positive; otherwise, it outputs zero. ReLU is widely used due to its simplicity and efficiency. It helps mitigate the vanishing gradient problem but can suffer from the "dying ReLU" problem, where neurons can become inactive and stop learning if they output zero consistently.

$$\text{ReLU}(x) = \max(0, x)$$

Weight Initialization

1) Random Initialization

Random initialization involves setting the initial weights to small random values. This helps break symmetry and ensures that neurons learn different features.

Weights are initialized randomly, often from a uniform or normal distribution.

$$W_{ij} \sim U(-a, a)$$

where U is the uniform distribution and a is a small constant.

$$W_{ij} \sim N(0, \sigma^2)$$

where N is the normal distribution with mean 0 and variance σ^2

2) Xavier (Glorot) Initialization

Xavier initialization aims to keep the scale of gradients roughly the same across all layers. It works well with activation functions like sigmoid and tanh.

For a layer with n_{in} input neurons and n_{out} output neurons, the weights are initialized as:

$$W_{ij} \sim U\left(-\sqrt{\frac{6}{n_{in} + n_{out}}}, \sqrt{\frac{6}{n_{in} + n_{out}}}\right)$$

$$W_{ij} \sim N\left(0, \frac{2}{n_{in} + n_{out}}\right)$$

3) He Initialization (Kaiming Initialization)

He initialization is designed for layers with ReLU activation functions that takes into account the non-linearity of activation functions.. It helps maintain the variance of activations through the layers.

For a layer with n_{in} input neurons, the weights are initialized as:

$$W_{ij} \sim U\left(-\sqrt{\frac{6}{n_{in}}}, \sqrt{\frac{6}{n_{in}}}\right)$$

$$W_{ij} \sim N\left(0, \frac{2}{n_{in}}\right)$$

Dropout Regularization

Dropout is a regularization technique used to prevent overfitting in neural networks. It works by randomly "dropping out" a fraction of the neurons during each training iteration.

During training, each neuron's output is set to zero with a probability p . This means that during each forward and backward pass, only a subset of the neurons is used, which prevents the network from becoming overly reliant on any particular neuron.

For each training example and each neuron, the output of neuron i in layer l is modified as follows:

$$Z_i^{(l)} = \begin{cases} 0 & \text{with probability } p \\ \frac{A_i^{(l)}}{1-p} & \text{with probability } 1-p \end{cases}$$

where $Z_i^{(l)}$ is the output of the i -th neuron in layer l after applying dropout, $A_i^{(l)}$ is the activation of the i -th neuron before applying dropout, and p is the dropout rate (probability of dropping a neuron).

During testing, all neurons are used, but their outputs are scaled by $1-p$ to maintain the overall output magnitude.

$$Z_i^{(l)} = A_i^{(l)}(1-p)$$

Loss Function

1) Mean Squared Error (MSE)

It is used for regression tasks. MSE is the average of the squared differences between the predicted and actual values.

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Where, N is the number of data points, y_i is the true value, and $\hat{y}$ is the predicted value.

2) Binary Cross Entropy Loss

$$BCE = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

3) Categorical Cross-Entropy Loss

Categorical cross-entropy loss, also known as softmax loss, is a loss function commonly used in multi-class classification problems. Before calculating the categorical cross-entropy loss, the outputs of the neural network are typically passed through a softmax activation function to convert them into a probability distribution.

For a batch of N samples, the average categorical cross-entropy loss is:

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c})$$

Where, $y_{i,c}$ is the true binary indicator (0 or 1) if class label c is the correct classification for i -th sample, $\hat{y}_{i,c}$ is the predicted probability of the i -th sample belonging to class c .

Gradient of Categorical Cross-Entropy Loss

During backpropagation, the gradients of the loss with respect to the logits are needed to update the weights. This gradient is used in optimization

algorithms like gradient descent to update the weights of the neural network. The gradient of the categorical cross-entropy loss with respect to the logits $z_{i,c}$ is:

$$\frac{\partial L}{\partial z_{i,c}} = \hat{y}_{i,c} - y_{i,c}$$

C. Instrumentation tools

The process was executed entirely using Python, with Google Colab as the primary platform for running. Jupyter Notebook served as our primary platform for coding and experimentation. Google Colab offers built-in functions for data analysis, making it ideal for the lab. The numpy library played a central role in handling numerical computations and array operations, facilitating the manipulation of input data and weight updates. The pandas library was utilized to organize and manage data in a tabular format, streamlining the preprocessing process. For visualizations, matplotlib.pyplot and Seaborn were employed to create various plots, graphs, and visualizations to better understand the model's performance. Evaluation metrics were computed using the scikit-learn library. The confusion matrix function was applied to analyze the accuracy of classification by comparing predicted labels with actual labels. These tools and libraries collectively streamlined data processing, facilitated model training, and allowed for a comprehensive analysis of the ANN model's performance through various metrics and visualizations.

D. Algorithm to train Neural Network from Scratch

This algorithm describes the procedure for training a neural network from scratch using gradient descent. It begins by initializing the neural network parameters, including weights and biases for input, hidden, and output layers. The training process consists of multiple epochs, during which each training example undergoes forward propagation to compute activations at each layer. Loss is then calculated using a specified loss function. Backpropagation is performed to

compute gradients, which are used to update the parameters through gradient descent. The algorithm iterates over these steps, adjusting the parameters to minimize the loss and improve the network's predictive accuracy, eventually returning the trained network parameters.

Require: Training data (X_train, y_train), Number of epochs num_epochs, Learning rate α , Number of hidden layers num_hidden_layers, Number of neurons per hidden layer num_neurons, Activation function activation_function, Loss function loss_function

Ensure: Trained neural network parameters

Initialize neural network parameters:

Randomly initialize weights and biases for input, hidden, and output layers.

for epoch = 1 to num_epochs do

for each training example (X, y) in (X_train, y_train) do

Perform forward propagation:

Set input layer values: $A^{[0]} = X$

for $l = 1$ to num_hidden_layers + 1 do

Compute $Z^{[l]} = W^{[l]}A^{[l-1]} + b^{[l]}$

Compute $A^{[l]} = \text{activation_function}(Z^{[l]})$

end for

Compute loss using loss_function(y, $A^{[\text{num_hidden_layers}+1]}$)

Perform backward propagation (Backpropagation):

Compute $dZ^{[\text{num_hidden_layers}+1]} = A^{[\text{num_hidden_layers}+1]} - y$

for $l = \text{num_hidden_layers} + 1$ to 1 do

Compute $\Delta W^{[l]} = \frac{1}{m} \Delta Z^{[l]} A^{[l-1]T}$

Compute $\Delta b^{[l]} = \frac{1}{m} \sum \Delta Z^{[l]}$

Compute

```

         $\Delta Z^{[l-1]}$ 
    =  $W^{[l]T} \Delta Z^{[l]}$  activationfunc_derivative( $Z^{[l-1]}$ )
    end for
    Update parameters using Gradient
    Descent:
    for  $l = 1$  to num_hidden_layers + 1
    do
        Update  $W^{[1]} = W^{[1]} + \alpha dW^{[1]}$ 
        Update  $b^{[1]} = b^{[1]} - \alpha db^{[1]}$ 
    end for
end for
return Trained neural network parameters (weights
and biases)

```

E. Data Description

The ‘Letter Recognition’ and ‘MNIST’ dataset was taken from the UC Irvine Machine Learning Repository. The Letter Recognition dataset is composed of 16 numerical attributes extracted from 20x16 pixel images of letters from the alphabet. Each image is processed to derive features such as horizontal and vertical positions, width, height, and pixel counts, among others, to facilitate the classification of letters from A to Z. On the other hand, the MNIST dataset includes a large collection of grayscale images depicting handwritten digits ranging from 0 to 9. Each image in the MNIST dataset is represented as a 28x28 pixel grid, providing a straightforward yet challenging task for digit recognition algorithms. Both datasets are pivotal in benchmarking algorithms and advancing research in optical character recognition (OCR) and related fields.

We initially loaded these datasets using the scikit-learn library. Then, we converted categorical values into numerical formats to make them suitable for use with an Artificial Neural Network (ANN) model. This conversion process involves transforming categorical data, such as labels or other non-numeric attributes, into numerical codes. After that, we split each dataset into training

and validation sets, adhering to an 80:20 ratio to ensure comprehensive model training while reserving a portion of the data to assess the model’s performance on unseen data

F. Data Analysis and Visualization

Plots of various kinds are used to illustrate the insights obtained from the dataset and model training. The dataset’s features and their instances are displayed as separate histogram plots for every feature. Horizontal bar plots are another way that the training data’s outcomes are displayed. A heatmap is used to show how various attributes are correlated. A confusion matrix is used to visualize the model’s performance and outcomes after it has been trained.

IV. RESULT

A. TRAINING ANN WITH RANDOM WEIGHT INITIALIZATION

The neural network was trained by initializing the weights and biases randomly using NumPy functions. This random initialization can sometimes lead to model underperformance and extended training times if the initial weights and biases are far from the optimal values. The model was trained for 50 epochs, achieving a training accuracy of over 95%. The accuracy graph in Figure 13 plotted against the number of epochs shows a rapid increase in accuracy during the early stages of training, which gradually slowed down. The validation accuracy consistently stayed close to the training accuracy, maintaining a performance above 90% towards the end of the training. Despite the strong performance, some instances were misclassified, highlighting areas for further improvement. The model used ReLU as the activation function throughout the training process.

B. TRAINING ANN WITH XAVIER INITIALIZATION

The model was trained on the same dataset for an equal number of epochs (50) with different initialization and activation function configurations. In this instance, Xavier initialization was used for the weights, and the

ReLU activation function was applied throughout the network. The accuracy graph in Figure 15 plotted against the number of epochs shows a steady increase, with both the training and validation accuracy curves reaching a plateau. The maximum validation accuracy achieved was approximately 94.6%. Unlike previous configurations, the Xavier initialization led to a smoother and more stable convergence, indicating a more effective learning process. The final model showed a significant improvement in performance, with a balanced and consistent accuracy across the Letter Recognition dataset.

C. TRAINING ANN WITH KAIMING INITIALIZATION

In this case, Kaiming initialization was employed for the weights, with the ReLU activation function maintained throughout the network. The model was trained using different configurations over 50 epochs, exploring various initialization and activation functions. The accuracy graph in Figure 17 shows a strong convergence for both training and validation sets, with a slight fluctuation observed in the validation accuracy curve. The model achieved a maximum validation accuracy of approximately 95.6%, indicating a robust training process. The use of Kaiming initialization led to a smoother learning curve and a more efficient training process, resulting in a higher and more stable accuracy compared to other initialization methods.

D. TRAINING ANN WITH REGULARIZATION

We also applied dropout regularization to the model to assess its performance. Dropout Regularization randomly turns off some neurons during training by setting their weights to zero. This helps the model learn to generalize better by not relying too heavily on any single neuron, reducing the risk of overfitting. Other configurations remained the same, utilizing only the ReLU activation function and a learning rate of 0.1. The Figure 11 shows the accuracy plot using Dropout Regularization. The training accuracy obtained from the model was approximately 55%,

with the accuracy curve starting to flatten around 20 epochs. The validation accuracy reached about 76%, demonstrating strong performance on the validation dataset. The accuracy curves for both training and validation exhibited typical behavior, with the validation accuracy increasing rapidly initially and stabilizing around 40 epochs.

E. TRAINING ANN WITH MULTIPLE HIDDEN LAYERS

The ANN model was evaluated using two different configurations, one with 5 hidden layers and the other with 10 hidden layers. The Figure 19 shows confusion matrix for model with 5 hidden layers. It achieved an accuracy of 94.88%. Similarly, Figure 20 shows the confusion matrix for model with 10 hidden layers. The model with 10 hidden layers slightly outperformed it with an accuracy of 95.4%. The confusion matrices for both configurations show that most predictions fall along the diagonal, indicating correct classifications, with very few misclassifications. Notably, the 10 hidden layer model displayed fewer off-diagonal elements, which means it made fewer mistakes and had better classification accuracy and precision. The better performance of the 10-hidden-layer model suggests that having more hidden layers helps the model classify the data more accurately, though the improvement is only slight.

V. DISCUSSION AND ANALYSIS

First we plotted the graphs to illustrate the performance of neural networks with different activation functions and configurations, all utilizing He initialization. Each graph plots the accuracy of the training and validation sets over 50 epochs.

Figure 7 depicts the performance of a neural network using the Hyperbolic Tangent (Tanh) activation function. The training accuracy rapidly increases in the initial epochs and continues to improve, eventually approaching near-perfect accuracy of 95.37%. The validation accuracy follows a similar trend but plateaus slightly lower than the training accuracy, indicating a good but not perfect generalization to unseen data. Figure 6

presents the performance of a more complex neural network with ReLu, also using He initialization. The training accuracy improves quickly and approaches accuracy of 95.28%, similar to the Tanh activation graph. The validation accuracy, however, shows a more noticeable plateau and slight fluctuations.

Figure 9 shows the performance with the Sigmoid activation function. Both the training and validation accuracies increase steadily, with the validation accuracy closely following the training accuracy throughout the epochs. The model achieves an accuracy of 92.57% but does not reach the same peak performance as the Tanh activation. The close alignment between training and validation accuracies indicates better generalization and less overfitting compared to the Tanh activation.

The choice of activation function and model complexity significantly impacts the trade-off between training performance and generalization. While Tanh and more complex architectures can achieve higher training accuracies, they are more prone to overfitting, whereas Sigmoid activation offers a better balance between training accuracy and generalization.

The model's another configuration introduced Kaiming and Xavier initialization. The accuracy graph in Figure 17 indicates that Kaiming initialization, specifically designed for ReLU activation functions, provides an effective starting point for weight initialization, leading to fast convergence and high accuracy of 95.63%. In the case of Xavier initialization, the training accuracy of 94.6%. as shown in Figure 15. Although Xavier initialization is typically better suited for activation functions like sigmoid and tanh, it performs similarly well with ReLU activation in this Letter recognition dataset.

When using random initialization, the training accuracy shows a slightly slower convergence compared to Kaiming and Xavier initialization. After 50 epochs, the training dataset achieved an accuracy of approximately 94%. The accuracy

graph in Figure 13 exhibited an initial steep increase followed by a slowdown, indicative of learning convergence. The resulting confusion matrix in Figure 14 showed a commendable performance, although with some misclassifications. This demonstrates that while random initialization can still lead to good performance, it is not as effective as Kaiming or Xavier initialization for this setup.

The results indicate that increasing the number of hidden layers in the neural network leads to a gradual improvement in accuracy. The network with 2 hidden layers achieved an accuracy of 94.675%, while increasing the number of hidden layers to 5 slightly improved the accuracy to 94.875%. Further increasing the complexity of the network to 10 hidden layers results in a more noticeable boost in performance, reaching an accuracy of 95.4%. The confusion matrix can be visualized in Figure 19 and Figure 20. This trend suggests that deeper networks have a greater capacity to capture and learn complex patterns in the data, thereby enhancing their predictive performance. However, the incremental improvements from 2 to 5 layers and from 5 to 10 layers indicate that there may be diminishing returns with respect to adding more layers, as each additional layer contributes progressively smaller gains in accuracy. This emphasizes the importance of balancing model complexity with computational efficiency and the risk of overfitting, especially when the performance gains become marginal. In summary, the variations in model performance are due to the use of different activation functions and weight initialization methods. Although certain configurations performed better, they did not meet the anticipated results. These irregularities indicate that careful tuning and parameter adjustments are essential to reach the desired accuracy levels.

VI. CONCLUSION

To wrap it up, our exploration of different neural network setups using the Letter Recognition dataset showed us how important activation functions, weight initialization, and regularization

are for model performance. Starting with ReLU activation and random weight initialization gave us a good accuracy of about 94%, proving this basic setup works well for finding patterns. I.

Moreover, the journey through various configurations illuminated the intricacies involved in neural network tuning. It became clear that each component plays a crucial role in determining the model's accuracy and efficiency. We observed that seemingly minor adjustments, such as switching the activation function or modifying the weight initialization strategy, could lead to significant changes in performance. Testing various setups revealed a range of results. Using tanh activation slightly improved accuracy to 95.38%, whereas sigmoid activation reduced it to 92.58%. For weight initialization, Xavier Initialization yielded an accuracy of 0.946, ReLU Initialization achieved 0.95275, and Kaiming Initialization reached 0.95625. The introduction of dropout regularization resulted in an accuracy of 75.76%, underscoring the model's sensitivity to these changes and highlighting the need for careful selection and combination of techniques.

Although some configurations didn't perform as well as hoped, the models still achieved accurate predictions. For example, with regularization, the testing accuracy reached 75.76%, demonstrating that neural networks can tackle complex tasks, even if the results are not perfect.

In summary, our study highlights the importance of carefully and repeatedly adjusting neural networks. It has helped us understand how different choices affect performance and sets the

stage for future improvements and uses of neural networks.

REFERENCES

- [1] E. G. and M. Buscema, "Introduction to artificial neural networks," 2008.
- [2] F. ROSENBLATT, "THE PERCEPTRON: A PROBABILISTIC MODEL FOR INFORMATION STORAGE AND ORGANIZATION IN THE BRAIN," vol. 65, 1958.
- [3] D. E. Rumelhart, G. E. Hinton and R. J. Williams, "Learning representations by back-propagating errors," 1986.
- [4] L. Datta, "A Survey on Activation Functions and their relation with Xavier and He Normal Initialization," 2020.



KRISTINA BHANDARI is a computer engineering undergraduate student at IOE, Thapathali Campus. Her goal is to leverage her knowledge and expertise to make a significant impact in the field. (THA077BCT022)



NAMITA BHATTA is a computer engineering undergraduate currently in her fourth year at IOE, Thapathali Campus. She is eager to explore the latest advancements in AI (THA077BCT048)

APPENDIX

FIGURES

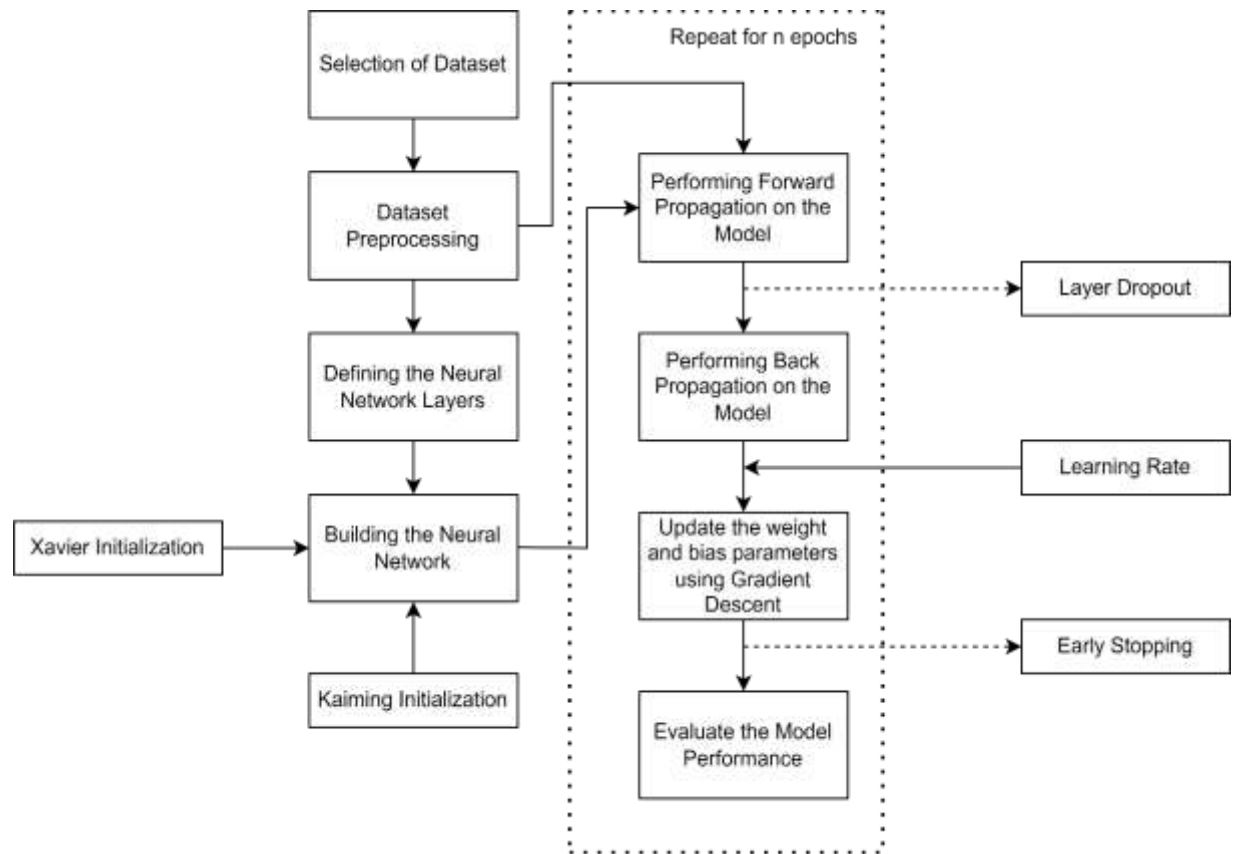


Figure 1 System Block Diagram

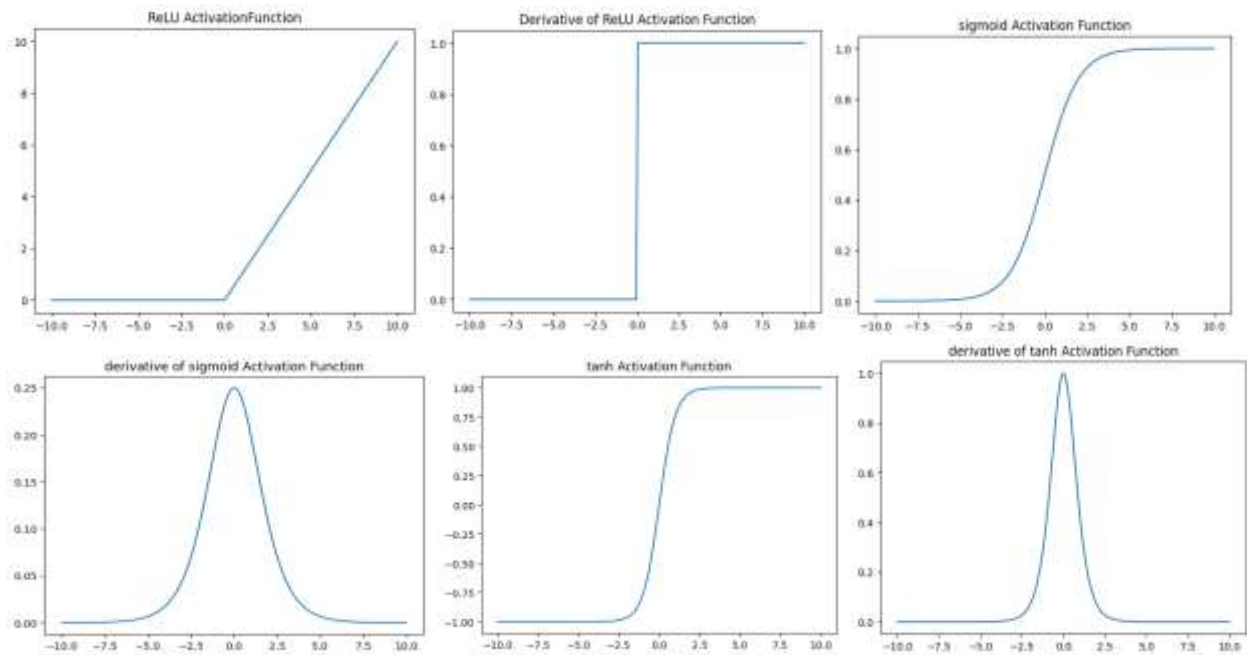


Figure 2: Plot of various activation functions and their derivatives

MNIST Dataset

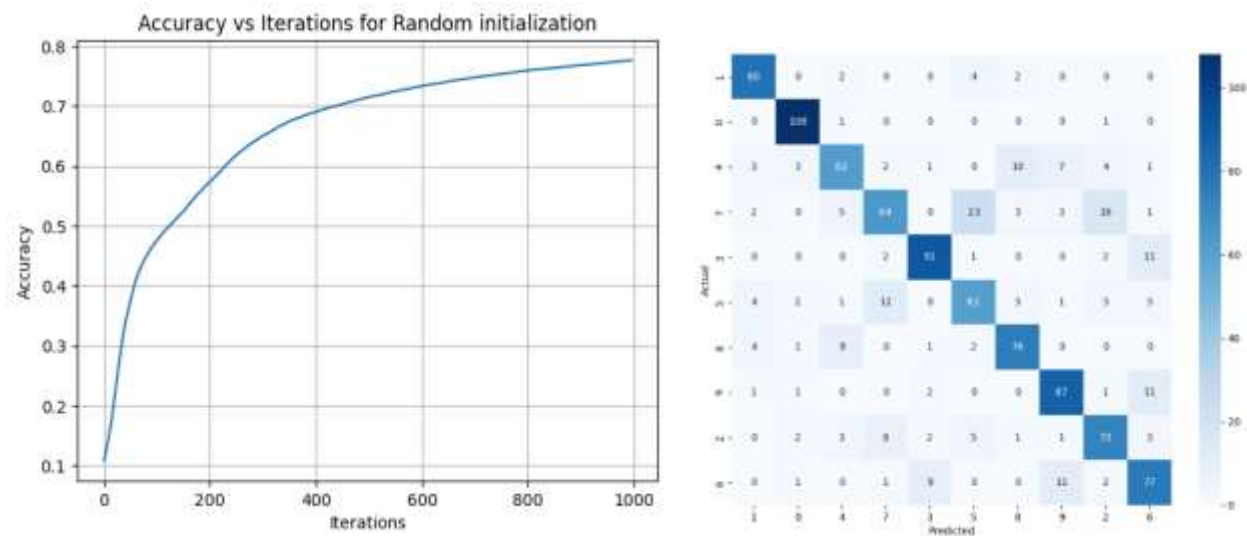


Figure 3: Plots and confusion matrix for Random Initialization

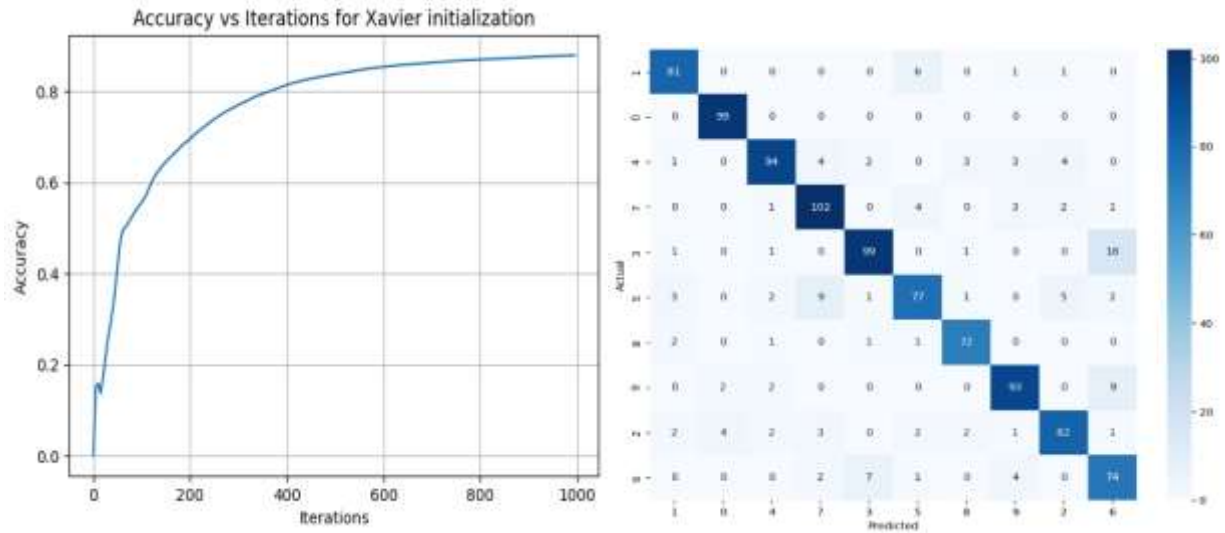


Figure 4: Plots and confusion matrix for Xavier Initialization

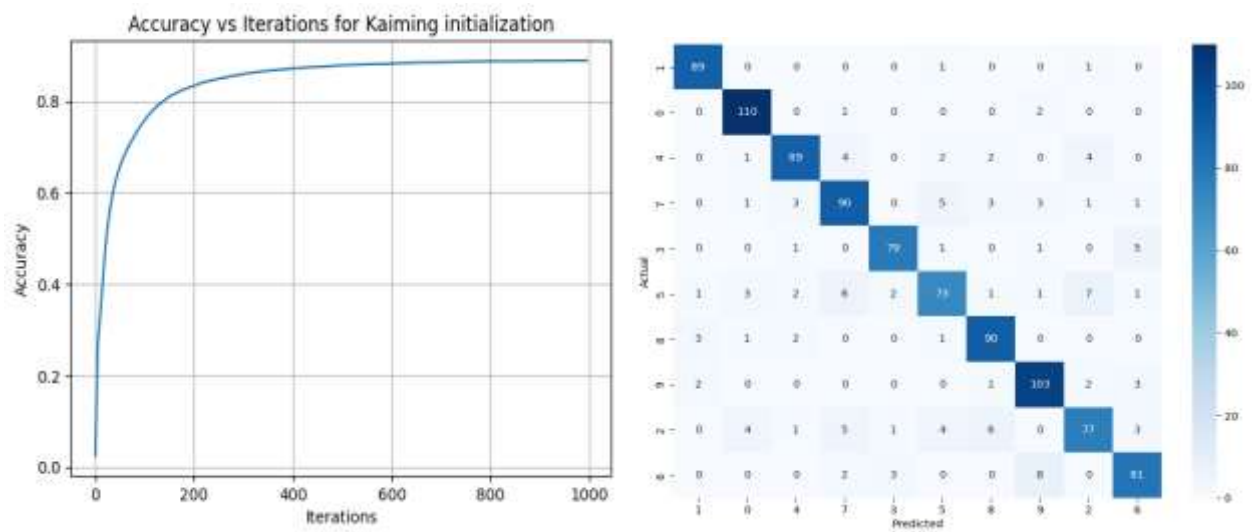


Figure 5: Plots and confusion matrix for Kaiming Initialization

Letter Recognition Dataset

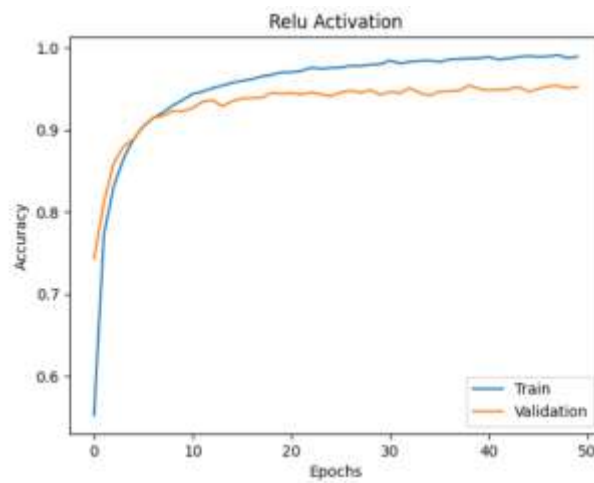


Figure 6: Accuracy vs Epoch graph for ReLu Activation function

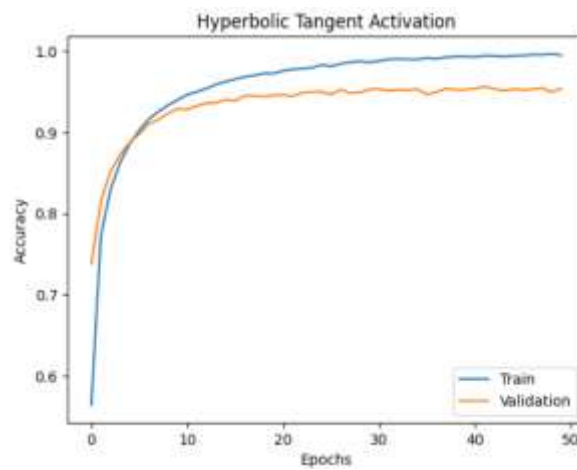


Figure 7: Accuracy vs Epoch graph for tanh Activation function

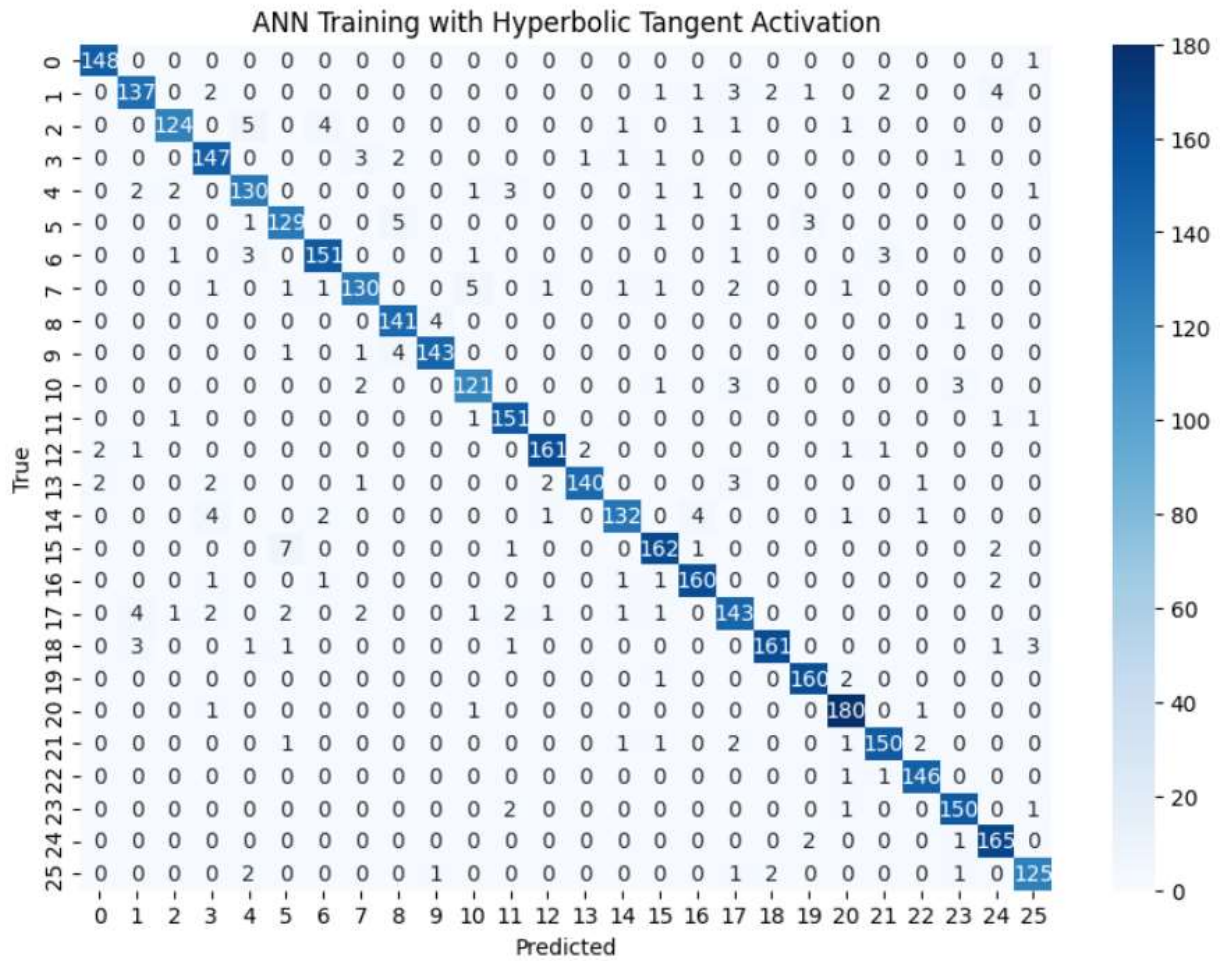


Figure 8: Confusion matrix while using Tanh activation

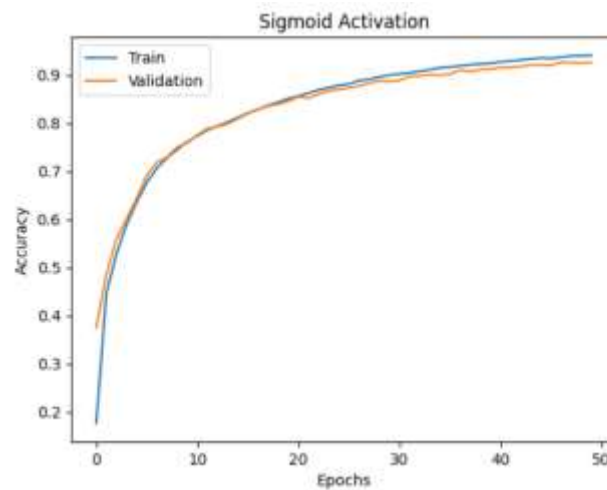


Figure 9: Accuracy vs Epoch graph for sigmoid Activation function

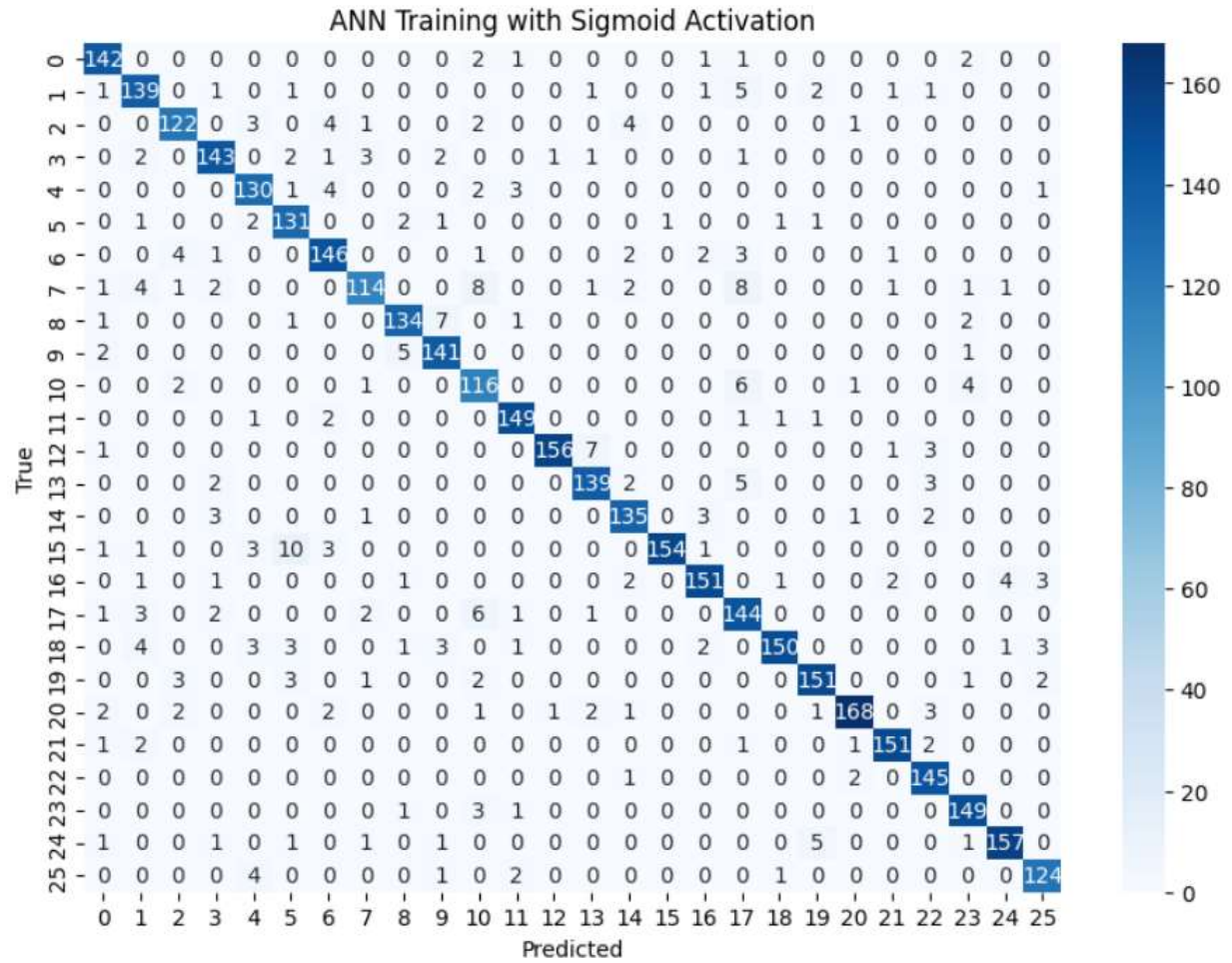


Figure 10: Confusion matrix for sigmoid Activation function

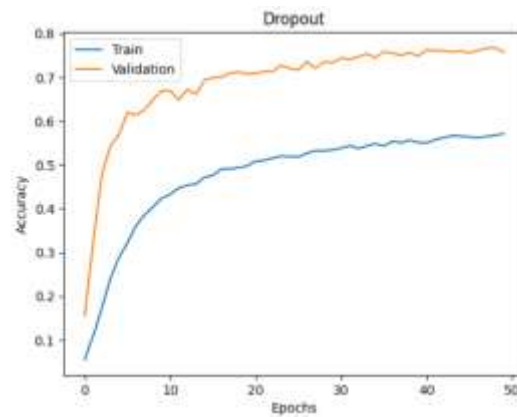


Figure 11: Accuracy vs Epoch graph with dropout

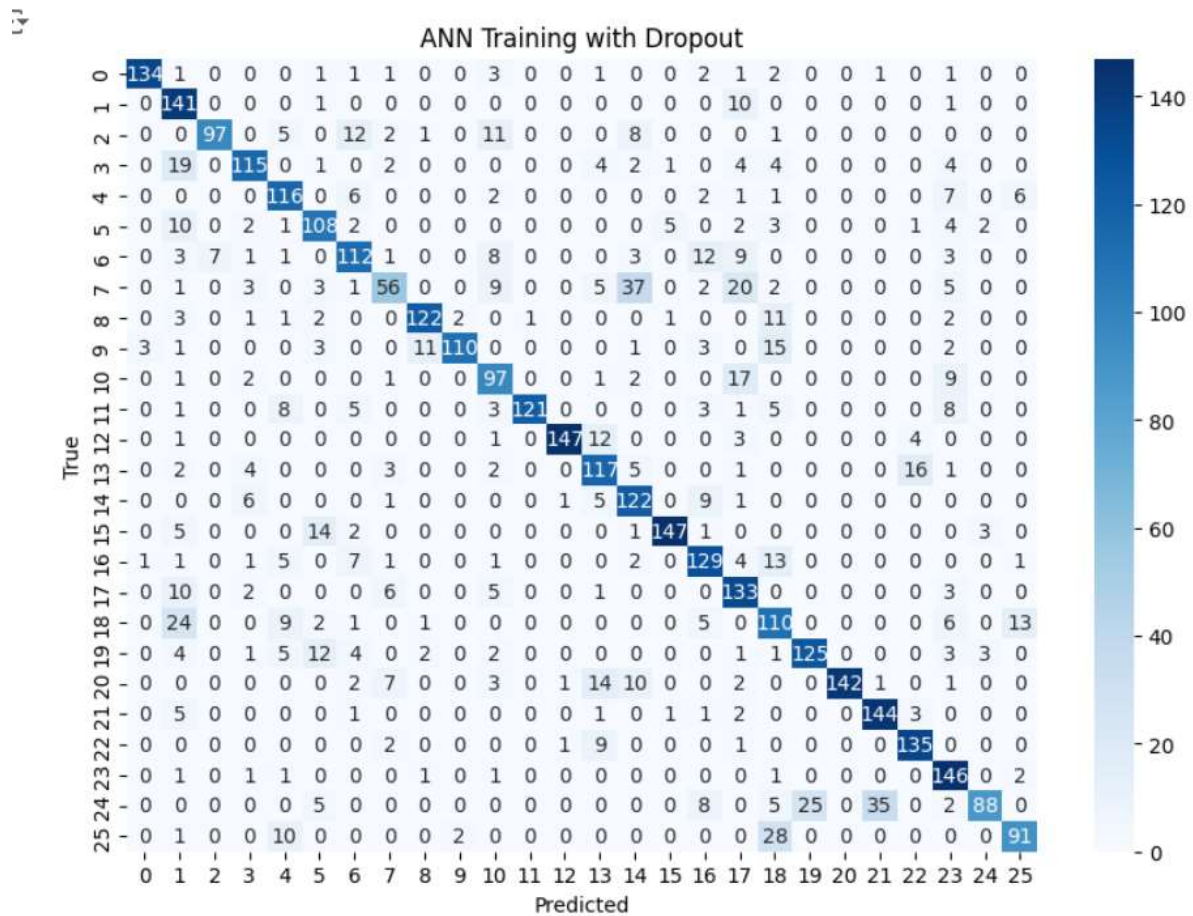


Figure 12: Confusion matrix while using dropout

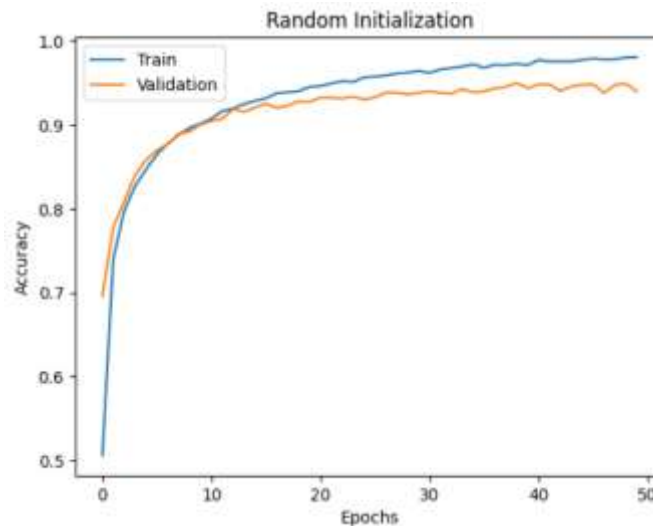


Figure 13: Accuracy vs Epoch graph with random initialization of weights

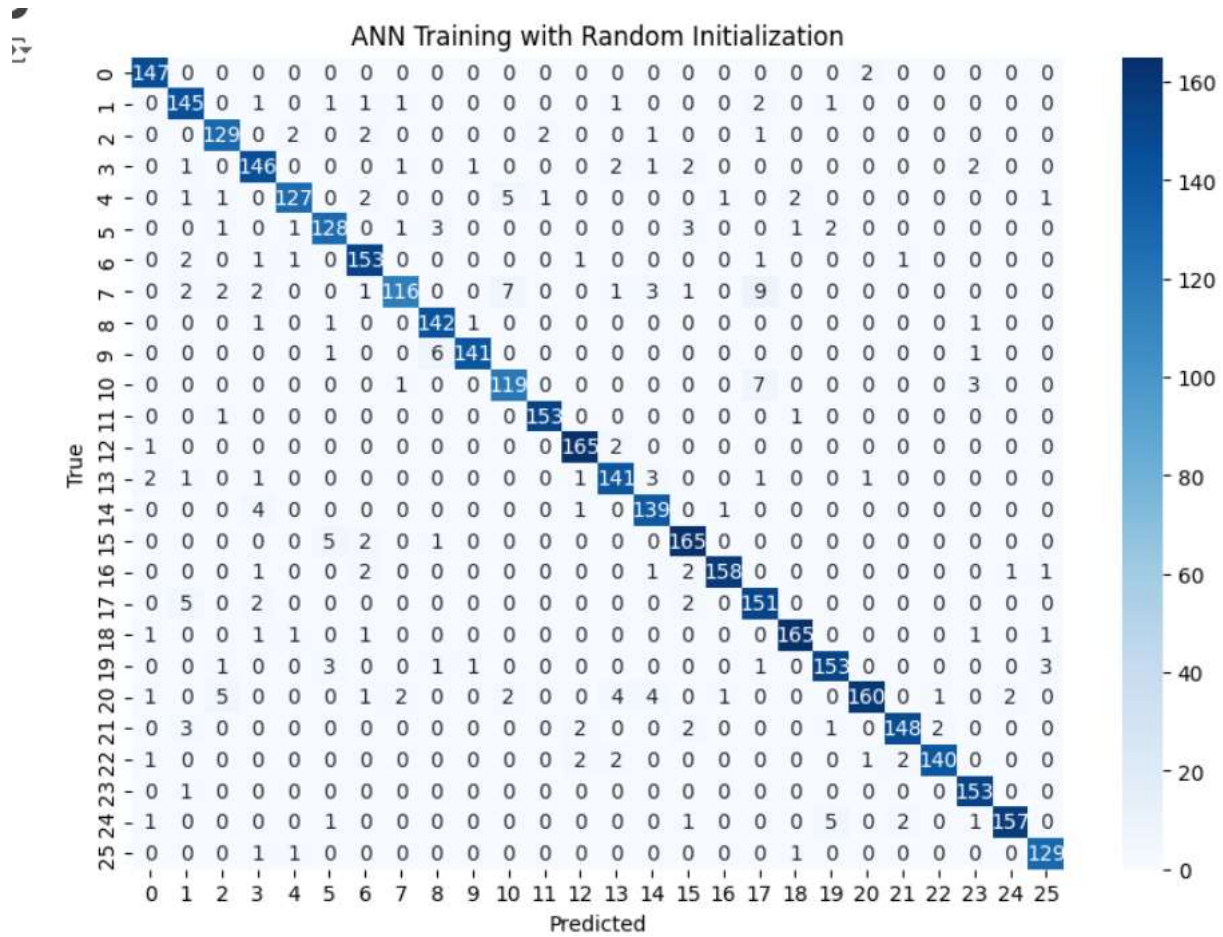


Figure 14: Confusion matrix with random initialization of weights

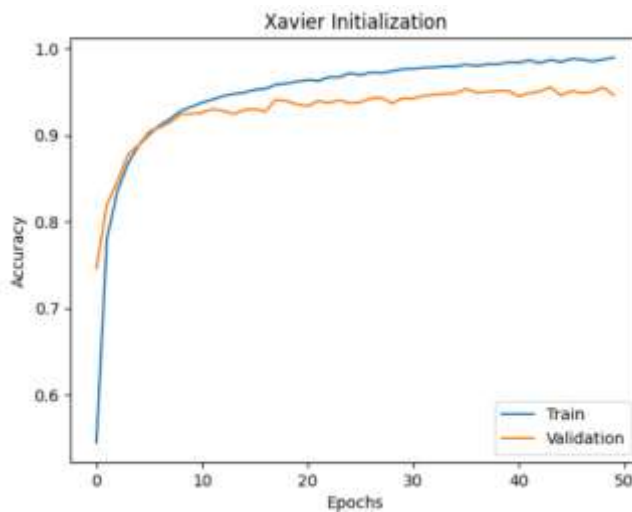


Figure 15 Accuracy vs Epoch graph with Xavier initialization

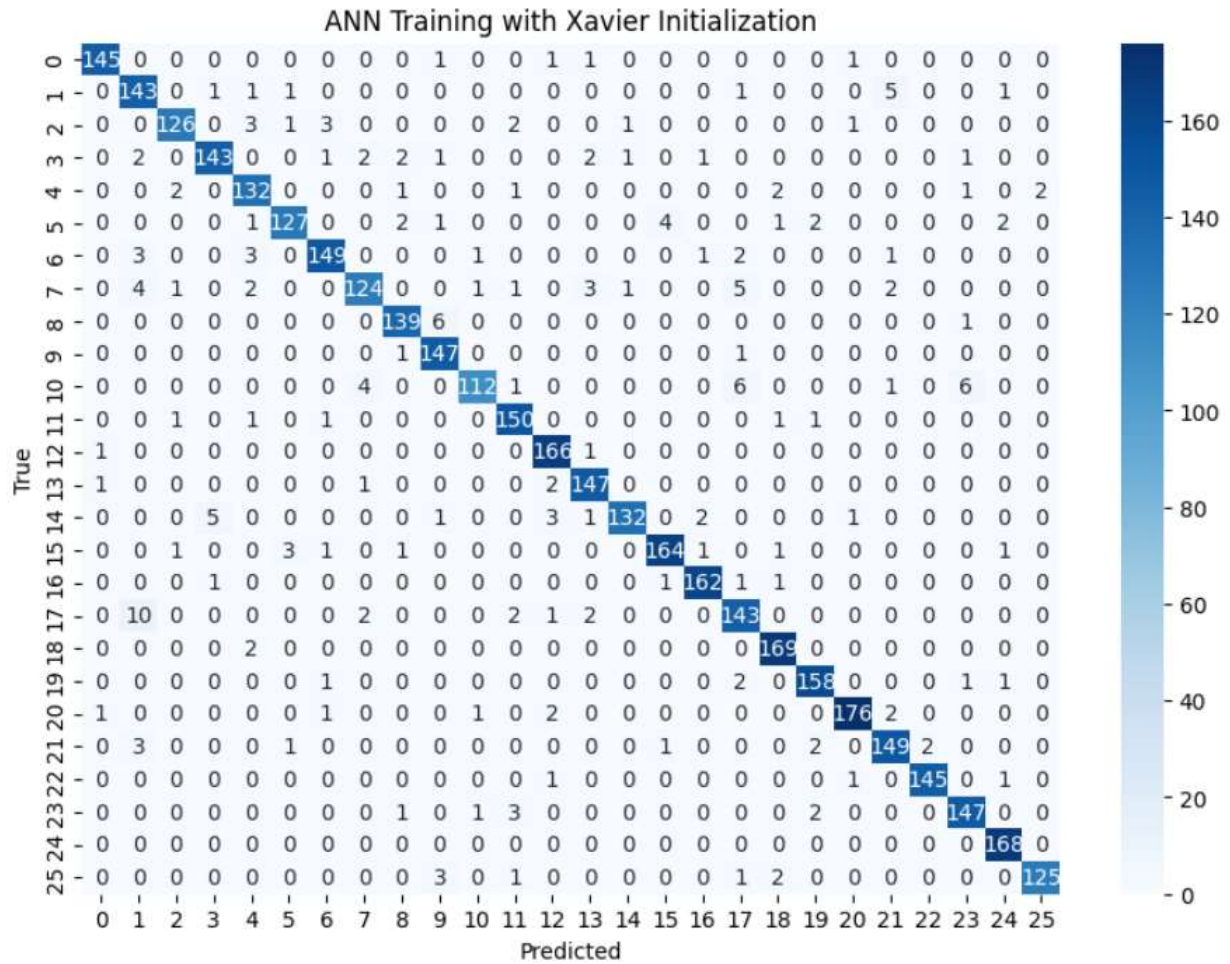


Figure 16: Confusion matrix with Xavier initialization

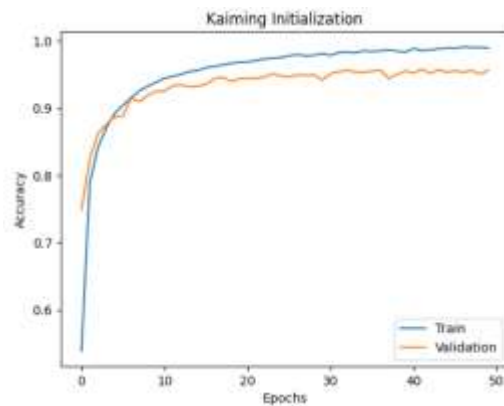


Figure 17: Accuracy vs Epoch graph with Kaiming initialization

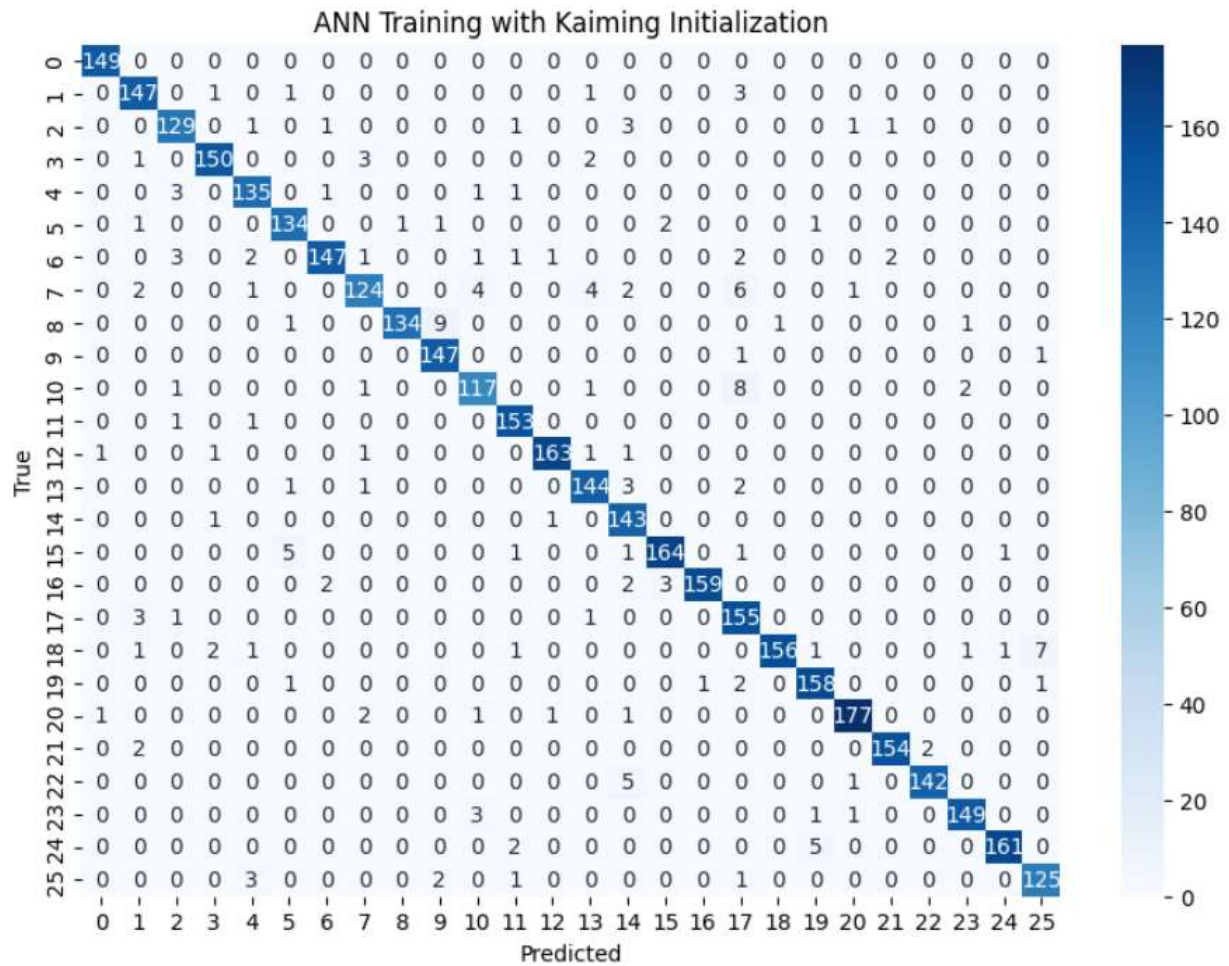


Figure 18: Confusion matrix with Kaiming initialization

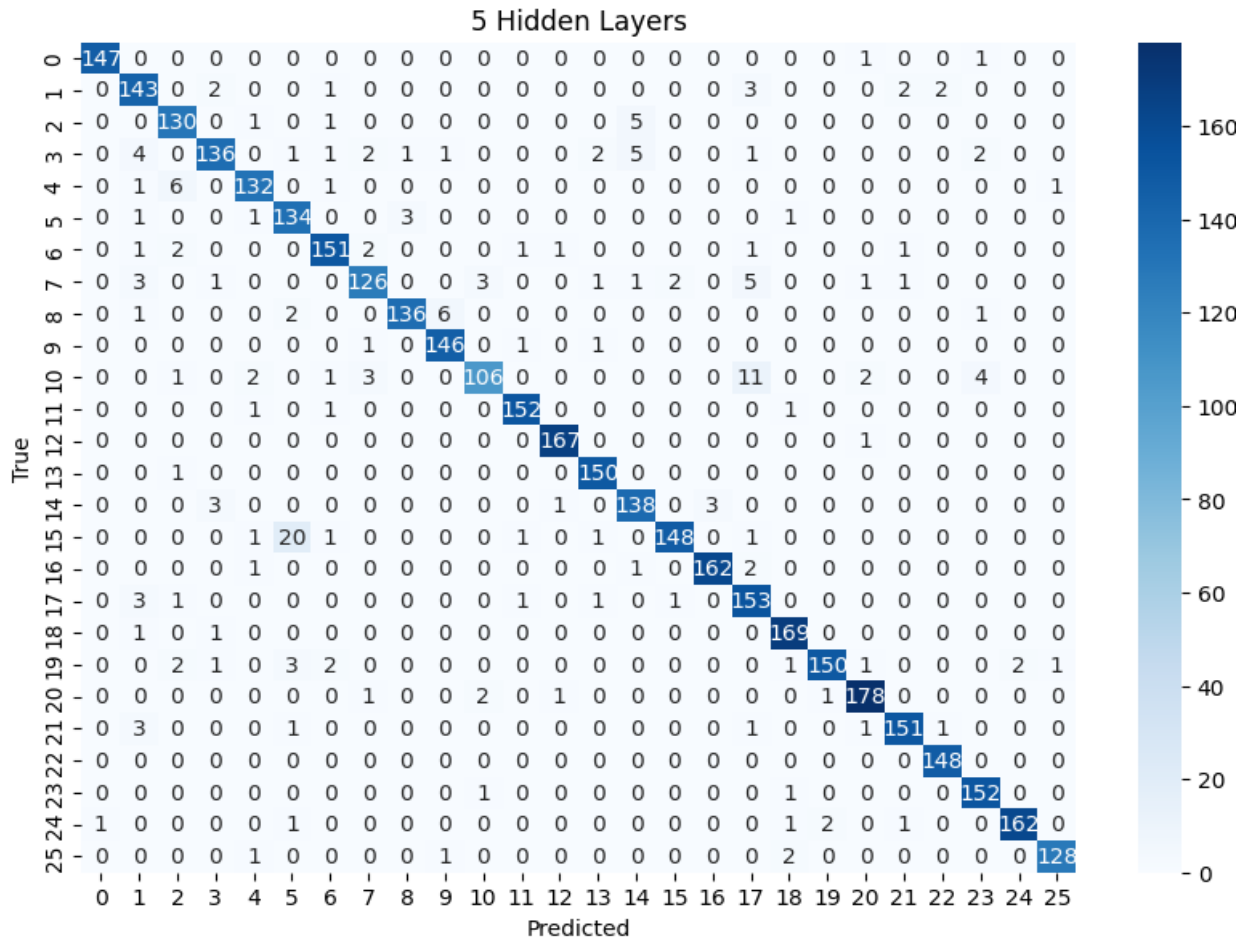


Figure 19: Confusion matrix for ANN training with 5 hidden layers

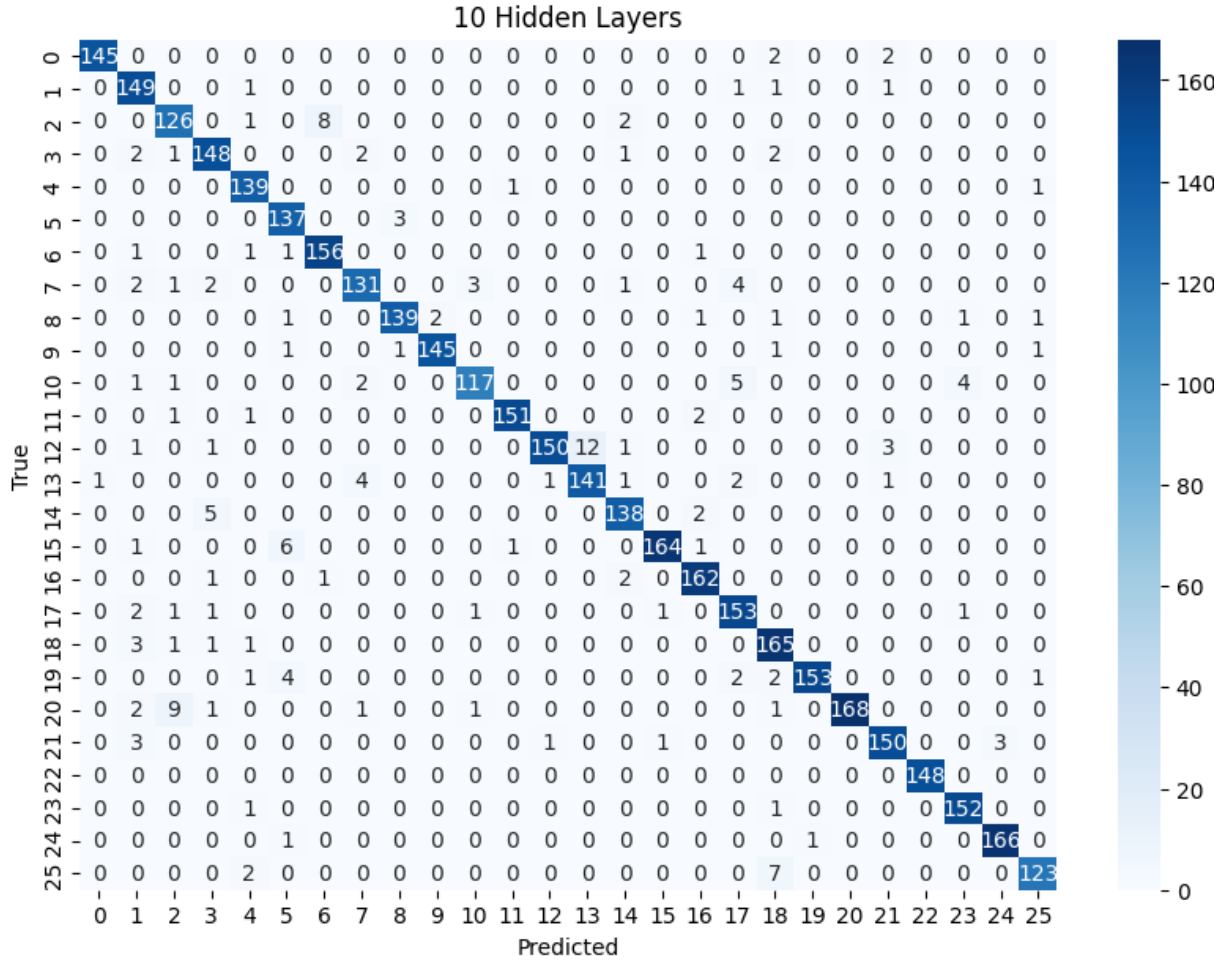


Figure 20: Confusion matrix for ANN training with 10 hidden layers

Backpropagation Derivation

Consider a simple neural network with one hidden layer. Let's denote the input as x , the hidden layer weights as W_h , the hidden layer bias as b_h , the hidden layer activation function as g , the output layer weights as W_o , and the output layer bias as b_o .

The forward propagation equations are:

$$Z_h = W_h \cdot x + b_h$$

$$A_h = g(Z_h)$$

$$Z_o = W_o \cdot A_h + b_o$$

$$A_o = g(Z_o)$$

Assume we have a loss function L that measures the error between the predicted output A_o and the target output y . We want to update the weights and biases to minimize the loss function using

gradient descent. The backpropagation algorithm computes the gradients of the loss with respect to the network's parameters.

The gradient of the loss with respect to the output layer activations:

$$\delta_o = \frac{\partial L}{\partial A_o} \cdot g'(Z_o)$$

The gradients of the loss with respect to the output layer weights and bias:

$$\frac{\partial L}{\partial W_o} = \delta_o \cdot A_h^T$$

$$\frac{\partial L}{\partial b_o} = \delta_o$$

The gradient of the loss with respect to the hidden layer activations:

$$\delta_h = (W_o^T \cdot \delta_o) \cdot g'(Z_h)$$

The gradients of the loss with respect to the hidden layer weights and bias:

$$\frac{\partial L}{\partial W_h} = \delta_h \cdot x^T$$

$$\frac{\partial L}{\partial b_h} = \delta_h$$

Finally, the weights and biases are updated using the gradients and the learning rate α :

$$W_o = W_o - \alpha \cdot \frac{\partial L}{\partial W_o}$$

$$b_o = b_o - \alpha \cdot \frac{\partial L}{\partial b_o}$$

$$W_h = W_h - \alpha \cdot \frac{\partial L}{\partial W_h}$$

$$b_h = b_h - \alpha \cdot \frac{\partial L}{\partial b_h}$$

This process of computing gradients and updating weights is repeated iteratively until convergence.